Cidades ESG Inteligentes

Desafio DevOps • Aplicação Java Spring Boot • Documentação técnica

Autora: Marina Espirito Santo Lopes — RM 558200. Projeto individual.

Objetivo: acompanhar iniciativas municipais ambientais, sociais e de governança, com cadastro, consulta, edição e exclusão.

Entrega: código-fonte, Dockerfile, Compose, pipeline GitHub Actions, scripts, README, evidências e checklist.

Validação local: oito testes executados, sem falhas ou erros; aplicação e PostgreSQL saudáveis no Docker. Persistência após recriar contêineres confirmada pela autora.

Nuvem: dois Web Services no Render; dados em schemas distintos do mesmo PostgreSQL. Ambientes acadêmicos de staging e produção.

Pipeline de CI/CD

GitHub Actions + deploy hooks do Render

1. Push em main ou execução manual inicia build-test em Ubuntu: mvn compile, mvn verify, relatório JUnit e docker build.

2. Staging: POST ao hook com ref igual ao SHA. Render constrói o Dockerfile e disponibiliza o serviço.

3. Smoke tests aguardam ambiente correto, commit exato, healthcheck UP e resposta da API. Timeout de 25 minutos.

4. Produção depende do sucesso de staging e repete o deploy do mesmo commit. Jobs dependentes não executam após falha.

Segredos: RENDER_DEPLOY_HOOK em cada GitHub Environment. Variável APP_URL por ambiente. Credenciais do banco ficam no Render.

Execução registrada: Success; build-test, staging e production concluídos. Duração total exibida: 3 min 32 s.
Não há rollback automático.

Docker e arquitetura

Imagem em duas etapas • Compose local • serviços gerenciados na nuvem

Dockerfile: Maven 3.9.9 / JDK 21 compila o JAR; Temurin JRE 21 executa como usuário sem privilégios. Curl verifica /actuator/health.

Imagem local: cidades-esg:local. O Compose cria app e db, volume dados e rede bridge esg. Banco sem porta publicada; aplicação local em 127.0.0.1:8080.

Comandos usados: docker run ... mvn -B verify; docker compose up -d --build --wait; docker compose down; docker compose up -d --wait.

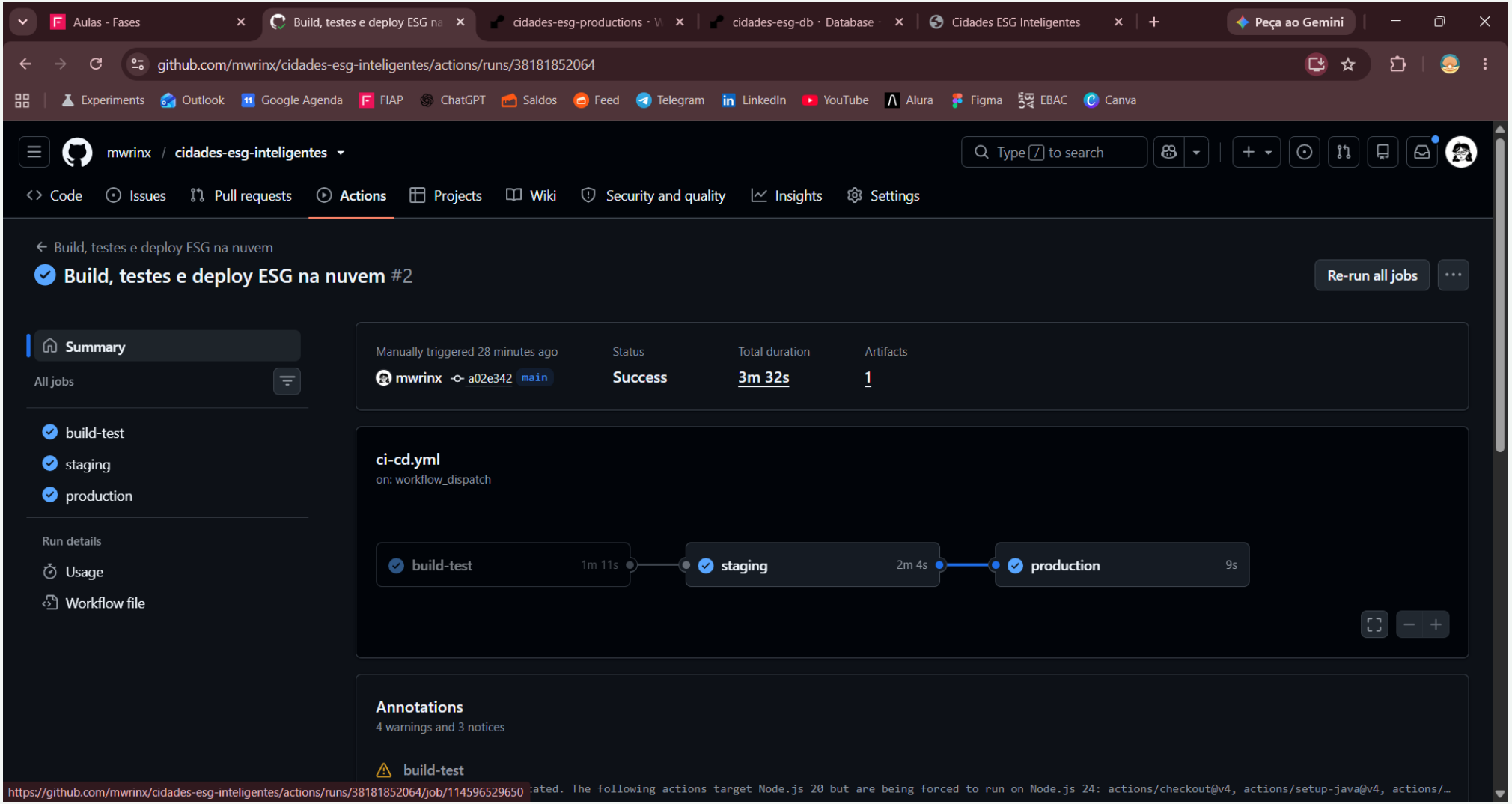
Render: dois serviços Docker e um PostgreSQL. APP_ENV identifica o ambiente; DB_SCHEMA seleciona staging ou production. PORT é fornecida pela hospedagem.

Schemas separam tabelas logicamente; usuário e servidor são compartilhados. O plano gratuito não representa isolamento completo de produção.

Build GitHub valida Dockerfile; Render reconstrói cada imagem pelo commit. Não há promoção do mesmo digest de imagem entre ambientes.

Evidência: pipeline concluído

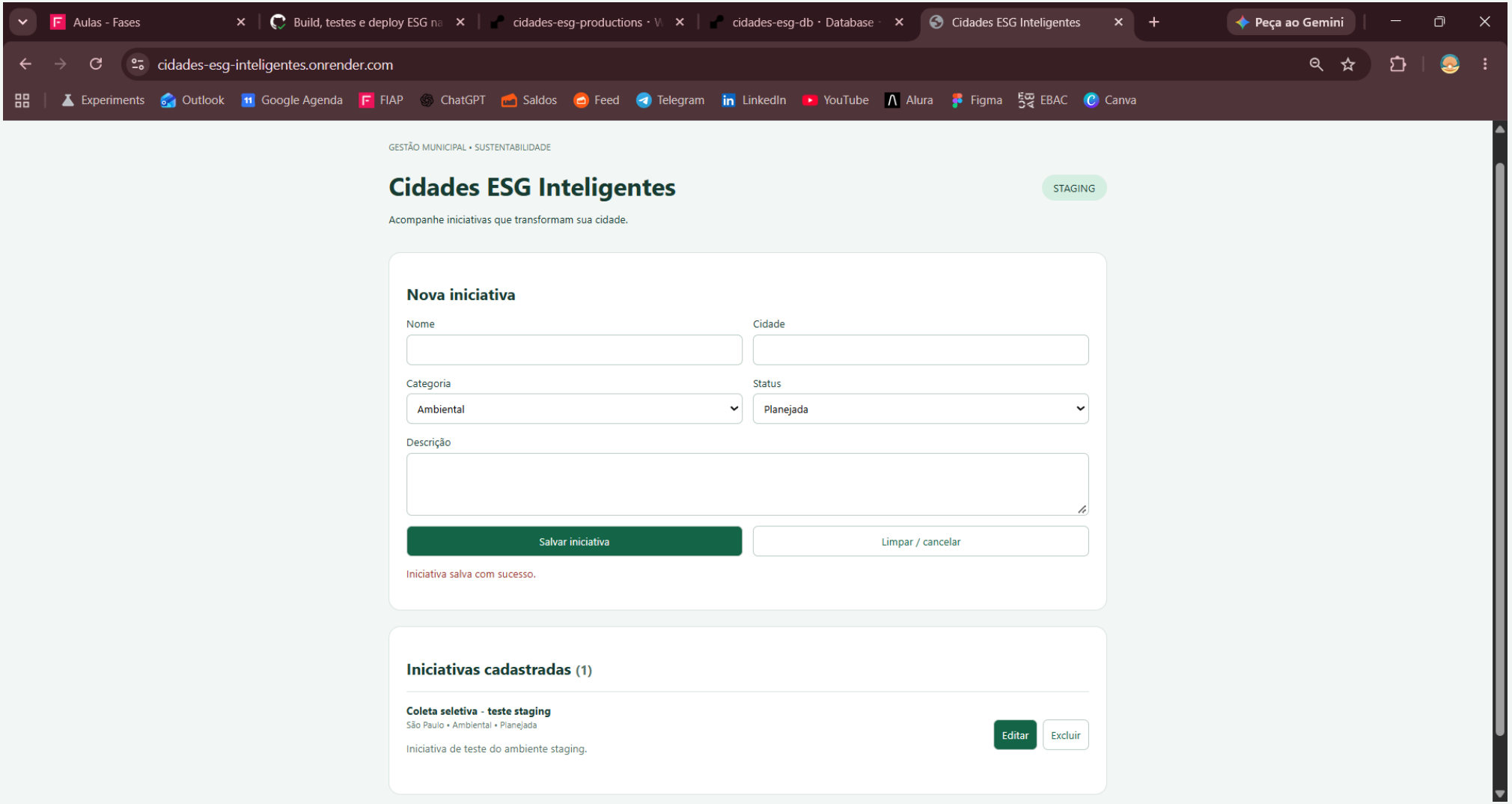
GitHub Actions • resultado Success • três jobs concluídos



Execução: github.com/mwrinx/cidades-esg-inteligentes/actions/runs/38181852064

Evidência: staging

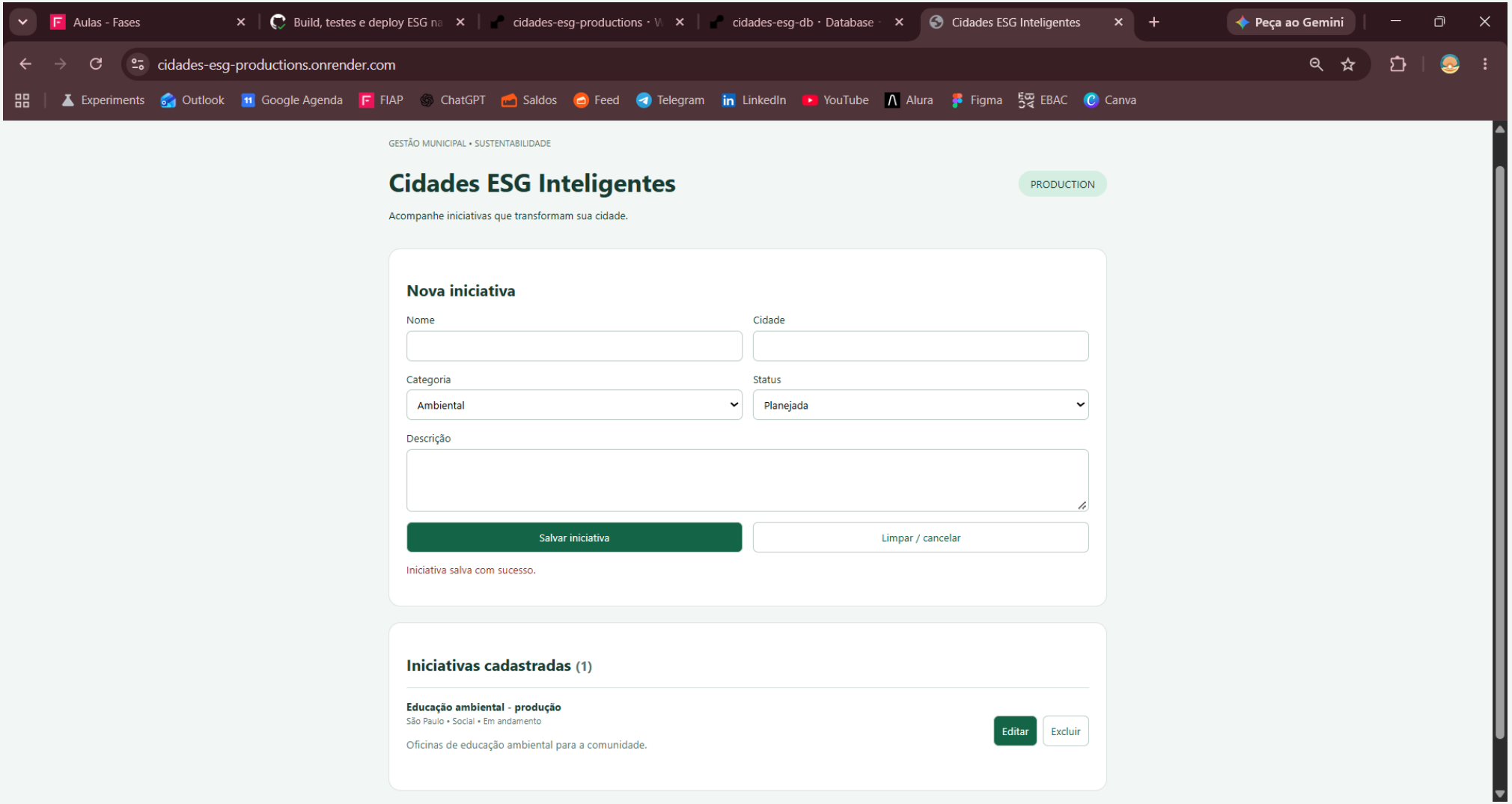
Iniciativa ambiental cadastrada • indicador STAGING



https://cidades-esg-inteligentes.onrender.com/

Evidência: produção

Iniciativa social cadastrada • indicador PRODUCTION



https://cidades-esg-productions.onrender.com/

Desafios e soluções

Correções realizadas durante a implementação

WSL / Virtual Machine Platform ausentes: instalados e habilitados; reinício do Windows permitiu executar o Docker.

Device Guard bloqueou runner local: adotados runners hospedados pelo GitHub e deploy em nuvem; proteções do Windows preservadas.

Erro database does not exist: corrigido Database no endereço JDBC. Erro password authentication failed: corrigidas credenciais de produção.

Plano gratuito aceita um PostgreSQL: usados dois schemas, com separação lógica de dados. Prints mostram cadastros distintos em cada ambiente.

Actions emitiram avisos de depreciação, mas a execução terminou com sucesso. Atualização das versões permanece como melhoria futura.

Limites: suspensão por inatividade; banco gratuito expira em 30 dias; sem autenticação ou backups neste exercício. Usar somente dados fictícios.

Checklist de entrega

Conferência dos artefatos obrigatórios

OK	Projeto compactado em ZIP com estrutura organizada
OK	Dockerfile funcional
OK	docker-compose.yml com volumes, rede e variáveis
OK	Pipeline executado: build, testes e deploy
OK	README com instruções e prints
OK	Documentação técnica PDF com evidências
OK	Deploy em staging e produção realizado

Repositório privado: prints incluídos permitem avaliar as evidências sem acesso ao GitHub.

ZIP exclui .env, segredos, .git e target; inclui .env.example e o PDF.